13-Bridge Pattern

- The **Bridge Pattern** is a structural design pattern in Software Engineering that separates an abstraction from its implementation so the two can vary independently.

- Bridge Pattern យើងគិតវាដូចជាស្ពាន់ចឹង គឺវាមានតួនាទីអោយវាធ្វើការតភ្ជាប់គ្នាពីទិសដៅខុសគ្នា ចឹង យើងត្រូវគិតថា ត្រូវធ្វើ មួយ អោយដូចអាមួយទៀត។ គឺត្រូវបង្កើត Abstract class យើងមិនអាចបង្កើត object from abstract class បានទេ។ តែយើងអាចអោយ class ណាដែល extends from abstract class ធ្វើការបង្កើត បាន។
- ចឹង មុនបង្កើត Object instance យកទៅប្រើប្រាស់បានអោយវាធ្វើការ បង្កើត Object មុនសិន។ ឧ អោយវាបង្កើត Object សម្រាប់ bank ABC first or XYZ first before it create.

```java
package com.vibol.corepattern.brige13;

public class TestBankPayment {

    public static void main(String[] args) {
        ABCBank abcBank = new ABCBank();
        XYZBank xyzBank = new XYZBank();

        UPIPayment upiPayment = new UPIPayment(abcBank);
        upiPayment.makePayment("abcl123", 1000);

//      CardPayment cardPayment = new CardPayment(xyzBank);
//      cardPayment.makePayment("xyz123", 2000);

    }

}
```

Using UPI Payment

Processing Payment of 1000.0 Through ABC Bank for the account abcl123

យើងចង់ ផ្ទេរលុយពី ABC Bank to XYZ bank. ចឹងយើងអោយវាបង្កើត Object ABC Bank សិន។

🧠 One-Line Memory Trick

👉 "Bridge = connect two things so they can change independently."

🌉 Imagine a Real Bridge

Think of a **bridge between two islands**:

- Left side = **Abstraction** (what you control)
- Right side = **Implementation** (how it works)
- Bridge = connection between them

👉 You can change either side **without breaking the other**

🧠 One-Line Difference (Easy to Remember)

👉 **Bridge = design for flexibility (before building)**
👉 **Adapter = fix incompatibility (after building)**

🧠 Final Memory Trick

👉 Say this:

- **Bridge = "I plan for change"**
- **Adapter = "I fix mismatch"**

- Bridge = ធ្វើការផ្លាស់ប្តូរមុន នឹងបង្កើត។ bank payment with different bank
- Adapter = វាខុសគ្នាចឹងបានយើងត្រូវធ្វើការបំលែងអោយវា ទៅជាប្រភេទតែមួយ

🧠 Final Memory (Stick This)

👉 **Bridge**

- You design it early
- "I want flexibility"

👉 **Adapter**

- You fix it later
- "I want compatibility"

🔥 Super Simple Way to Tell in Interview

If you see:

- ❓ "Too many combinations" → ✅ **Bridge**
- ❓ "Cannot connect / incompatible" → ✅ **Adapter**

1- Create interface for different bank payment

```java
package com.vibol.corepattern.brige13;

public interface Bank {

    void processPayment(String accountNumber, double amount);

}

```

ទិន្នន័យរបស់ Bank ត្រូវការ លេខគណនី នឹងទឹកលុយ ដើម្បីធ្វើការ ផ្ញើរទៅ other bank។ តែយើងត្រូវដឹងថា គេផ្ញើរតាមអី Credit payment or UPI payment វាជាប្រភេទ លក្ខណៈដែលធ្វើការដក់ប្រាក់ចេញពី ទិន្នន័យ Account នឹង

2- ABC bank implementation Bank

```java
package com.vibol.corepattern.brige13;

public class ABCBank implements Bank {

    @Override
    public void processPayment(String accountNumber, double amount) {
        System.out.println("Processing Payment of " + amount +
                " Through ABC Bank for the account " + accountNumber);
    }

}
```

3- XYZ bank implementation Bank

```java
package com.vibol.corepattern.brige13;

public class XYZBank implements Bank {

    @Override
    public void processPayment(String accountNumber, double amount) {
        System.out.println("Processing Payment of " + amount
                + " Through XYZ Bank for the account " + accountNumber);
    }

}
```

4- Bridge: Payment យើងគិតថាដើម្បីធ្វើការ ផ្ទារប្រាក់បានត្រូវការអ្វីខ្លះ(abstract class)

- Create object Bank នឹងសិន it is ABC or XYX bank
- យើងត្រូវការអ្វីខ្លះដើម្បីធ្វើការផ្ញើរទៅបាន

```java
package com.vibol.corepattern.brige13;

public abstract class Payment {

    protected Bank bank;

    public Payment(Bank bank) {
        this.bank = bank;
    }

    public abstract void makePayment(String accountNumber, double amount);

}
```

5- Payment class ដើម្បីធ្វើការកំណត់ Class ដែលបង្កើតពីវា

```java
package com.vibol.corepattern.brige13;

public abstract class Payment {

    protected Bank bank;

    public Payment(Bank bank) {
        this.bank = bank;
    }

    public abstract void makePayment(String accountNumber, double amount);

}
```

6- Card Payment យើងធ្វើការ បង់លុយតាម card

```java
package com.vibol.corepattern.brige13;

public class CardPayment extends Payment {

    public CardPayment(Bank bank) {
        super(bank);

    }

    @Override
    public void makePayment(String accountNumber, double amount) {
        System.out.println("Using Credit Card Payment");
        bank.processPayment(accountNumber, amount);

    }

}
```

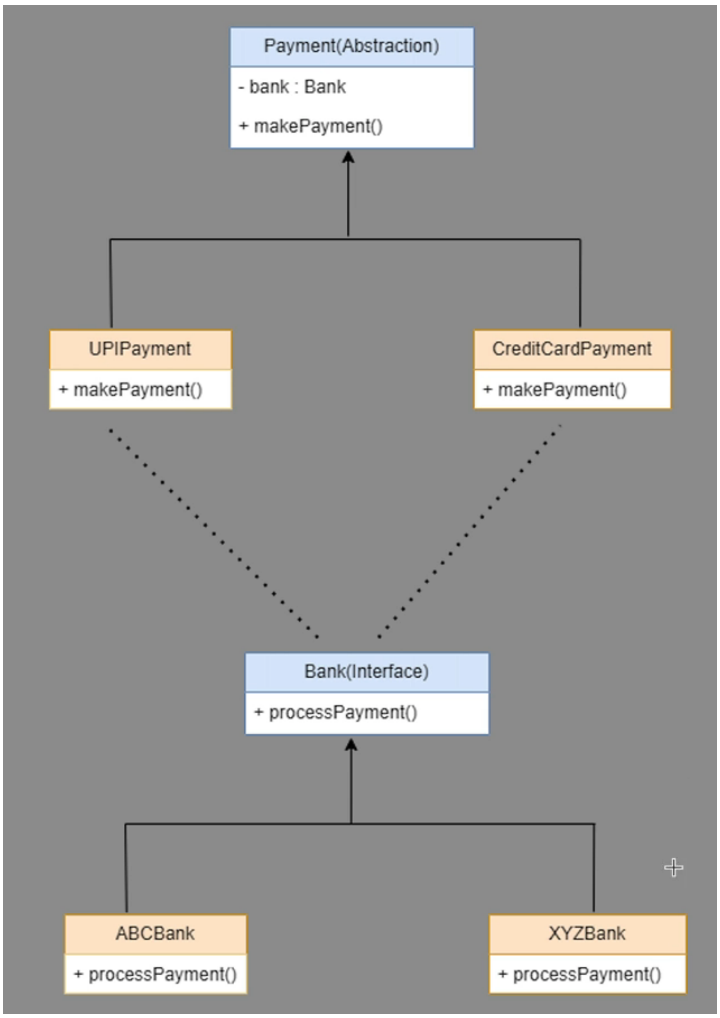
យើងបង្កើត Constructor ដែលត្រូវអោយបង្កើតមកមាន Object Bank នឹងតែម្តងៗដែល Bank វាជា ABC or XYZ bank

7- XYZ bank

```java
package com.vibol.corepattern.brige13;

public class UPIPayment extends Payment {

    public UPIPayment(Bank bank) {
        super(bank);

    }

    @Override
    public void makePayment(String accountNumber, double amount) {
        System.out.println("Using UPI Payment");
        bank.processPayment(accountNumber, amount);

    }

}
```

8- Testing

```java
package com.vibol.corepattern.brige13;

public class TestBankPayment {

    public static void main(String[] args) {
        ABCBank abcBank = new ABCBank();
        XYZBank xyzBank = new XYZBank();

        UPIPayment upiPayment = new UPIPayment(abcBank);
        upiPayment.makePayment("abcl123", 1000);

        CardPayment cardPayment = new CardPayment(xyzBank);
        cardPayment.makePayment("xyz123", 2000);

    }

}
```

Problems @ Javadoc Declaration Terminal Console Boot Dashboard

```
<terminated> TestBankPayment [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (Apr 27, 2026, 1:42:07 PM – 1:42:08 PM) [pid: 21012]
Using UPI Payment
Processing Payment of 1000.0 Through ABC Bank for the account abcl123
Using Credit Card Payment
Processing Payment of 2000.0 Through XYZ Bank for the account xyz123
```